

Deployment, Session & Configuration

1. How would you fix session handling so users don't get logged out when the app runs on multiple servers?

A: You fix this by setting up **shared session management**. You move the user session data out of the individual server's memory and into a **central, external store** like **Redis**. This way, if a user's next request hits a different server, that new server can still retrieve the correct session data from Redis.

2. How do you set up shared session management in a Spring Boot app running on multiple nodes?

A: You set it up by integrating **Spring Session** and configuring it to use a **Redis** or database backend. Spring automatically replaces the default local session handler with one that reads and writes session data to the shared external store, ensuring session stickiness across all nodes.

3. How do you safely roll back a production deployment when a critical bug goes live?

A: The safest way is to use a strategy like **Blue-Green Deployment**. Since the old, stable version of the app (Blue environment) is still running, you immediately **reroute 100% of the user traffic** from the buggy new version back to the stable old version. This provides an instant and safe rollback.

4. How do you protect API keys or database passwords during build and deployment?

A: You protect them by **never checking them into Git**. Instead, you store them in a secure **Secret Management** tool like **HashiCorp Vault** or **Kubernetes Secrets**. During deployment, the application reads the secrets directly from the secure store at runtime, ensuring they are never exposed in configuration files or build logs.

5. How do you manage database schema changes during deployments safely?

A: You manage schema changes using **database migration tools** like **Flyway** or **Liquibase**. These tools automatically track which schema versions have been applied and execute the

necessary SQL scripts only during the deployment process, ensuring all changes are versioned and applied safely before the application starts.

6. How do you achieve zero-downtime deployments?

A: You achieve zero-downtime deployments by using **rolling updates** or **Blue-Green deployment**. Rolling updates gradually replace old instances with new ones, keeping the service available. Blue-Green switches traffic instantly after the new environment is fully validated. Both methods ensure traffic is served continuously during the upgrade.

7. How do you diagnose and fix a performance issue introduced after deployment?

A: I would diagnose by checking **monitoring dashboards** (like Prometheus/Grafana) for spikes in CPU, memory, or latency. I would then take a **thread dump (jstack)** to see which threads are stuck or running slow code. The fix often involves tuning a database query, optimizing a tight loop, or increasing resource limits.

8. What is Blue-Green deployment, and when would you use it?

A: **Blue-Green deployment** involves running two identical environments (Blue is live, Green is the new version). Traffic is instantly switched from Blue to Green after testing. You would use it for critical applications where **zero downtime** and **instant rollback** capability are absolute requirements, as it is the safest deployment method.

9. How do you identify and fix bottlenecks in a deployment process that takes too long?

A: I identify bottlenecks by reviewing the **CI/CD pipeline logs** to measure the time taken by each stage (e.g., compile, run tests, build Docker image). The fix usually involves enabling **parallel builds** (using the `-T` flag in Maven), optimizing slow tests, or using **Cloud Native Buildpacks** to speed up Docker image creation.

10. Which monitoring tools do you use, and how do they help track deployment health?

A: I use **Prometheus** (for collecting metrics) and **Grafana** (for visualizing dashboards). They track health by showing key metrics like **Latency** (requests are fast), **Error Rates** (failed requests are low), and **CPU/Memory utilization** on the new deployment. They provide immediate feedback if the new version is unstable.

11. How would you investigate a memory leak in a running microservice after deployment?

A: I would investigate by using a tool like **VisualVM** or **JMX** to monitor heap usage remotely. If the memory keeps growing and never drops after Garbage Collection (GC), I would trigger a **Heap Dump** (a memory snapshot). I would then analyze the dump using **Eclipse MAT** (Memory Analyzer Tool) to find the object that is consuming memory and fix the code that holds onto it.

12. During rolling deployments, users see inconsistent behavior. What could be causing it?

A: Inconsistent behavior is typically caused by **API version drift** or **incompatible data changes**. Since both the old version and the new version of the code are running simultaneously during the rollout, they must be fully backward compatible. If the new version changes the API contract or data schema in a way the old version cannot handle, users hitting the old service see errors.